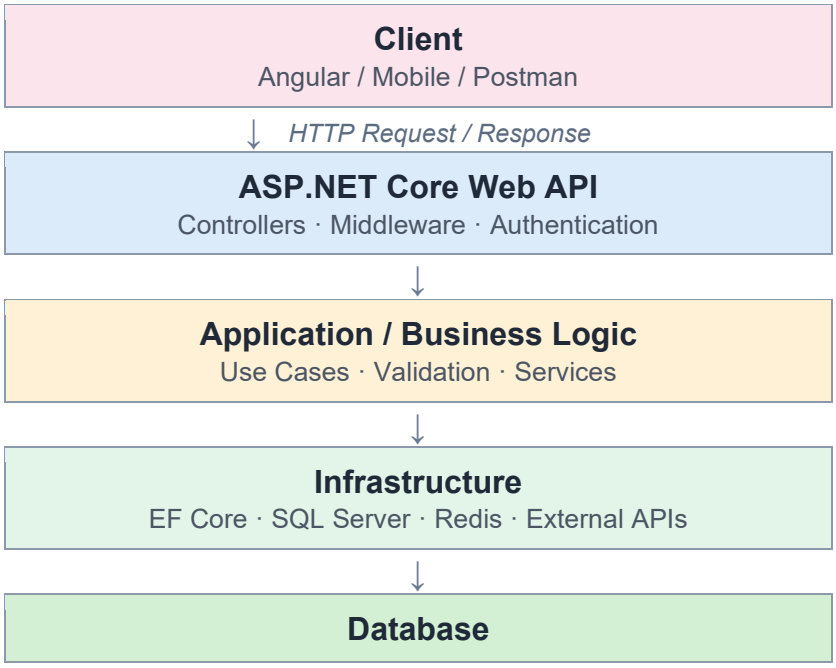
ASP.NET Core Architecture

شرح مبسّط: Architecture و Clean Architecture و CQRS مع MediatR

يعني إيه ASP.NET Core Architecture؟

كلمة **Architecture** معناها طريقة تصميم وتنظيم أجزاء التطبيق، وتحديد مسؤولية كل جزء وطريقة تواصله مع الأجزاء التانية.

تخيّل إن عندك API خاصة بمتجر إلكتروني، والعميل بيطلب المنتجات.



> **الفكرة الأساسية:** بدل ما تحط كل الكود في مكان واحد، بتقسمه لمسؤوليات واضحة، علشان التطبيق يكون أسهل في التطوير والاختبار والصيانة.

> **مهم:** ده مثال لتقسيم معماري ممكن تستخدمه، مش شرط كل ASP.NET Core API تحتاج الأربع طبقات دي.

إيه اللي بيحصل لما العميل يطلب API؟

لنفترض إن العميل بعت الطلب ده:

```
GET /api/products/10
```

الطلب بيمر بمراحل، ودي من أهم الحاجات اللي لازم تفهمها:

1. **Middleware Pipeline:** الطلب يدخل على سلسلة من الـ Middleware، زي معالجة الأخطاء، والـ Logging، والـ Authentication.
2. **Routing:** ASP.NET Core يحدد الـ Endpoint أو الـ Controller المسؤول عن الطلب.

3. **Model Binding & Validation:** لو فيه Parameters أو Request Body، يتم ربط البيانات بالـ C# Objects والتحقق منها حسب الإعدادات.
4. **Controller:** يستقبل الطلب ويوجّه التنفيذ للـ Use Case أو الـ Service المناسب.
5. **Business Logic:** يتم تنفيذ قواعد العمل، زي التأكد إن المنتج موجود ومتاح.
6. **Data Access:** يتم جلب المنتج من SQL Server عن طريق EF Core مثلًا.
7. **Response:** النتيجة ترجع للعميل غالبًا في صورة JSON مع HTTP Status Code مناسب.

> **خد بالك:** ده المسار المنطقي المعتاد؛ مش كل Request لازم يوصل للـ Database، لأن ممكن الـ Middleware يرفضه، أو الـ Cache يرجّع النتيجة.

مكونات ASP.NET Core الأساسية

أولًا: Program.cs

ده المكان اللي بتبدأ منه تهيئة التطبيق.

```
var builder = WebApplication.CreateBuilder(args);

// Register services
builder.Services.AddControllers();

var app = builder.Build();

// Configure middleware
app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

خلّي بالك من الفرق بين الجزأين:

- `builder.Services`: بتسجل الـ Services والاعتماديات اللي التطبيق هيحتاجها.
- `app.Use...`: بتحدد الـ Middleware اللي الطلب هيمر عليها.
- `app.MapControllers()`: بتربط الـ Controller Endpoints بنظام الـ Routing.
- `app.Run()`: بتبدأ تشغيل التطبيق واستقبال الطلبات.

Dependency Injection (DI)

بدل ما الـ Controller ينشئ الـ Service بنفسه باستخدام `new`، بتخلي ASP.NET Core يوفر له الـ Dependency.

```
builder.Services.AddScoped<IProductService, ProductService>();
```

والـ Controller يستقبلها عن طريق Constructor Injection:

```
public class ProductsController : ControllerBase
{
    private readonly IProductService _productService;

    public ProductsController(IProductService productService)
    {
        _productService = productService;
    }
}
```

ليه ده مهم؟

- تقليل الاعتماد المباشر بين الكلاسات.
- سهولة اختبار الكود باستخدام Mock أو Fake Implementations.
- سهولة تغيير التنفيذ بدون تعديل الـ Controller.

Middleware

الـ Middleware عبارة عن مكونات بتتعامل مع HTTP Request و Response داخل الـ Pipeline.

أمثلة:

- Exception Handling
- Authentication
- Authorization
- CORS
- Logging

الترتيب مهم؛ مثلًا Authentication لازم يحصل قبل Authorization في السيناريو المعتاد:

```
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

> Authentication بتحدد المستخدم، و Authorization بتحدد هل المستخدم مسموح له ينفذ العملية ولا لأ.

إزاي ننظم المشروع؟ Layers & Responsibilities

دي أهم نقطة في موضوع Architecture: كل جزء في التطبيق يكون ليه مسؤولية محددة.

هنستخدم مثال Products API.

> **1. Presentation Layer — API**
> Controllers, HTTP, Authentication, Request/Response DTOs
> مسؤولة عن استقبال الطلبات وإرجاع النتائج، مش عن كتابة كل قواعد العمل.

2. Application Layer

Use Cases, Services, Commands, Queries, Interfaces

مسؤولة عن تنسيق تنفيذ العمليات المطلوبة، زي إنشاء منتج أو جلب تفاصيله.

3. Domain Layer

Entities, Value Objects, Business Rules, Domain Events

فيها قواعد العمل الأساسية والكيانات المهمة في الدومين، خصوصًا في الأنظمة المعقدة.

4. Infrastructure Layer

EF Core, DbContext, Repositories, Redis, External Services

مسؤولة عن التفاصيل التقنية والتعامل مع قواعد البيانات والخدمات الخارجية.

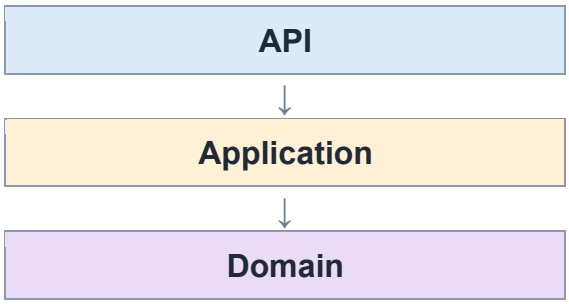
مثال على تنظيم الـ Solution

```
Products.sln
│
├── Products.API
│    ├── Controllers
│    ├── Middleware
│    ├── Program.cs
│    └── appsettings.json
│
├── Products.Application
│    ├── Interfaces
│    ├── DTOs
│    └── Products
│
├── Products.Domain
│    └── Entities
│
└── Products.Infrastructure
     ├── Persistence
     ├── Repositories
     └── Services
```

ده تنظيم مقترح لـ Clean Architecture، مش شكل إجباري لكل مشروع.

مين يعتمد على مين؟

في Clean Architecture، اتجاه الاعتماديات مهم جدًا:



الاعتماديات الداخلية لا تعتمد على تفاصيل الـ Infrastructure.

الـ Infrastructure تقدر تعتمد على الـ Application والـ Domain لتنفيذ الـ Interfaces المطلوبة، والـ API بتجمع الاعتماديات في الـ Composition Root، غالبًا داخل `Program.cs` أو ملفات تسجيل منفصلة.

مثال: الـ Application تعرّف `IProductRepository`، والـ Infrastructure تنفذها باستخدام EF Core.

```
// Products.Application
public interface IProductRepository
{
    Task<Product?> GetByIdAsync(
        int id,
        CancellationToken cancellationToken);
}
```

```
// Products.Infrastructure
public class ProductRepository : IProductRepository
{
    private readonly AppDbContext _context;

    public ProductRepository(AppDbContext context)
    {
        _context = context;
    }

    public Task<Product?> GetByIdAsync(
        int id,
        CancellationToken cancellationToken)
    {
        return _context.Products
            .FirstOrDefaultAsync(
                p => p.Id == id,
                cancellationToken);
    }
}
```

> لاحظ إن الـ Application مش محتاجة تعرف إن التنفيذ بيستخدم EF Core أو SQL Server.

Dependency Injection و Service Lifetimes

لازم تفهم الفرق بين أنواع الـ Lifetime:

النوع	معناه	مثال شائع
Transient	إنشاء Instance جديدة عند كل طلب من الـ Container	Service خفيفة لا تحتاج حالة مشتركة
Scoped	Instance واحدة داخل الـ Scope الحالي	DbContext في طلب HTTP
Singleton	Instance واحدة طوال عمر الـ Container	خدمة مشتركة آمنة مع تعدد الـ Threads

مثال:

```
builder.Services.AddTransient<IEmailService, EmailService>();
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddSingleton<ICacheManager, CacheManager>();
```

دي أمثلة وليست قاعدة ثابتة؛ اختيار الـ Lifetime يعتمد على طبيعة الخدمة واعتمادياتها.

الفرق بين Monolithic و Microservices

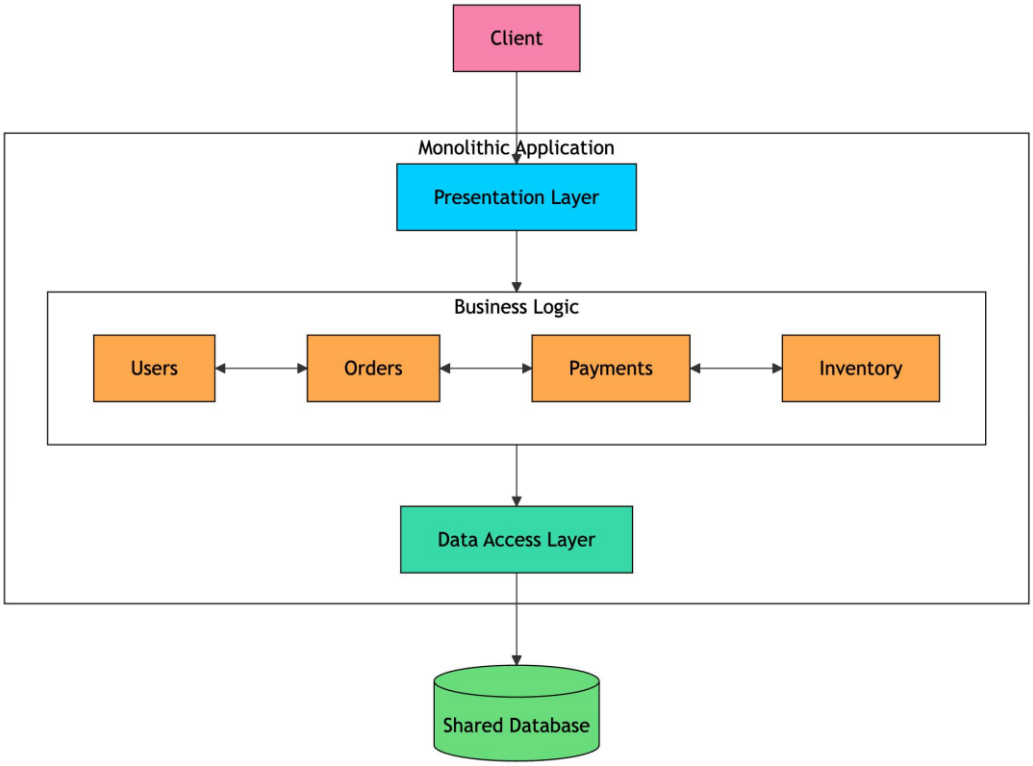
Monolithic	Microservices
تطبيق واحد يضم الـ API والـ Business Logic والـ Data Access. ممكن يكون منظمًا داخليًا لطبقات أو Features.	خدمات منفصلة تتواصل عبر HTTP أو Messaging، ويمكن نشر كل خدمة بشكل مستقل عند تصميم النظام لذلك.

مثال متجر إلكتروني:

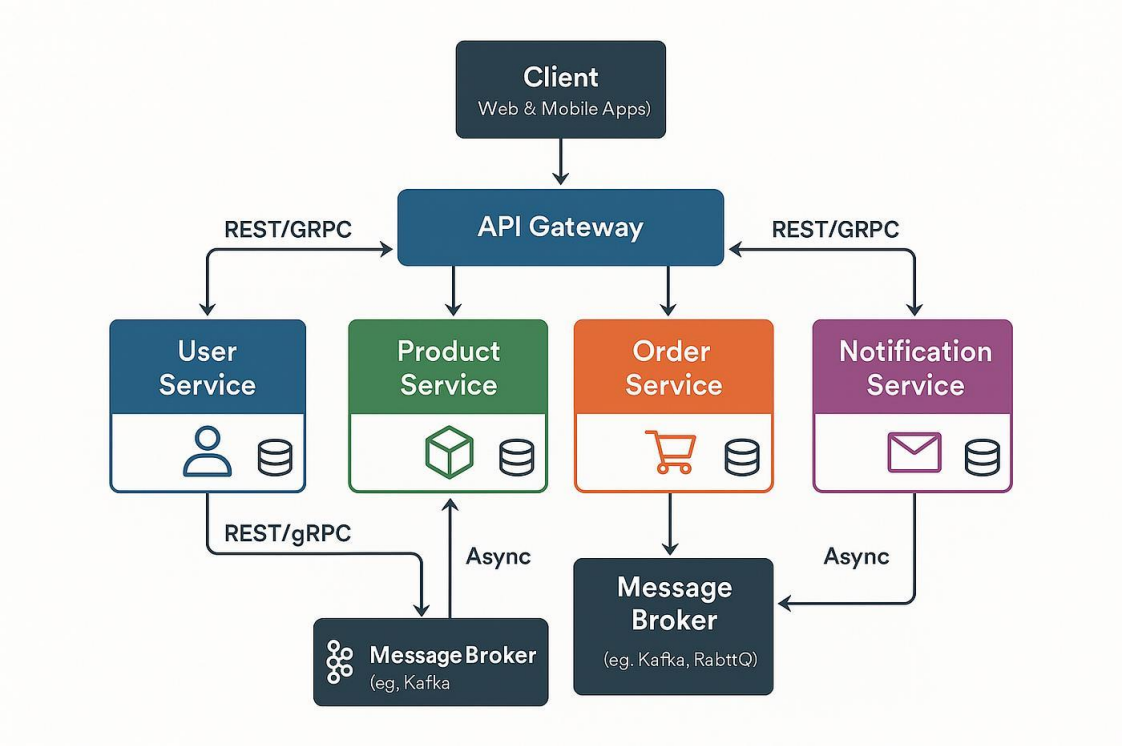
- Catalog Service للمنتجات.
- Basket Service للسلة.
- Ordering Service للطلبات.
- Discount Service للخصومات.

لو Basket محتاجة تطلب من Ordering إنشاء طلب، ممكن تتواصل معها مباشرة عبر HTTP، أو تنشر Event عن طريق RabbitMQ و MassTransit.

في الحالة الثانية، لازم تفهم إن Messaging بتضيف تحديات مثل تكرار الرسائل، وإعادة المحاولة، و Idempotency، واتساق البيانات بين الخدمات.



الشكل الأول: Monolithic Architecture



الشكل الثاني: Microservices Architecture

Architecture مش Layers بس: إيه كمان لازم تفهمه؟

في ASP.NET Core، فيه مفاهيم أساسية بتكمل الصورة.

Middleware Pipeline

فهم ترتيب الـ Middleware، ومعالجة الاستثناءات، والـ Authentication والـ Authorization، وإزاي توقف الطلب أو تكمل تمريره.

Dependency Inversion & DI

الاعتماد على Interfaces عند الحاجة، وإدارة الـ Lifetimes، وتجنب الـ Circular Dependencies.

Testing Architecture

Unit Tests لقواعد العمل، و Integration Tests للـ API وقاعدة البيانات، واختبار الـ Contracts المهمة.

Observability

Structured Logging باستخدام Serilog.

إزاي تفكر كـ Developer؟

مش مجرد شخص بيستخدم Layers أكتر؛ هو شخص بيختار التصميم المناسب ويفهم تكلفة قراراته.

أ. فصل المسؤوليات

لو الـ Controller فيه SQL Queries، و Validation، وحساب الخصم، وإرسال Email، يبقى غالبًا مسؤوليات كتير متجمعة في مكان واحد.

قسّم المسؤوليات حسب احتياج المشروع، من غير ما تنشئ Abstractions لكل سطر كود.

ب. معالجة الأخطاء

بدل ما تكتب `try/catch` متكرر في كل Controller:

- استخدم Centralized Exception Handling.
- سجّل الخطأ بمعلومات مفيدة باستخدام Serilog.
- ارجع Error Response مناسب، من غير كشف Stack Trace أو معلومات حساسة للعميل.
- ميّز بين أخطاء الـ Validation، والموارد غير الموجودة، والأخطاء غير المتوقعة.

ج. الأداء

- استخدم `async/await` مع عمليات الـ I/O.
- استخدم Pagination للقوائم الكبيرة.
- اختَر `AsNoTracking()` لقراءات EF Core التي لا تحتاج تتبع التعديلات.
- راقب N+1 Queries، وعدد استعلامات قاعدة البيانات.
- استخدم Caching لما يكون مناسبًا، مش لمجرد تحسين الأداء نظريًا.

د. الأمان

- افصل Authentication عن Authorization.
- تحقّق من الصلاحيات على مستوى المورد عند الحاجة، مش مجرد التأكد إن المستخدم مسجّل دخول.

- خزّن الأسرار في Configuration آمن، ولا تضعها في Git.
- لا تعتمد على بيانات العميل باعتبارها موثوقة.

هـ. قابلية الاختبار والصيانة

اسأل نفسك:

- هل أقدر أختبر قواعد العمل بدون تشغيل SQL Server؟
- هل تغيير طريقة التخزين هيجبرني أعدل قواعد العمل؟
- هل الـ API Contract واضح ومستقر؟
- هل أقدر أتتبع الطلب من أول دخوله لحد الخطأ؟
- هل التعقيد المعماري الموجود مبرر فعلًا؟

الأول: يعني إيه CQRS؟

اختصار لـ:

> Command Query Responsibility Segregation

بمعنى إننا بنفصل العمليات اللي بتغيّر البيانات عن العمليات اللي بتقرأ البيانات.

تخيّل إن عندك API فيها العمليات دي:

العملية	النوع
عرض كل الموظفين	Query
عرض موظف بالـ ID	Query
إضافة موظف	Command
تعديل موظف	Command
حذف موظف	Command

الفرق الأساسي:

- **Query:** بتقرأ البيانات فقط، ومش المفروض تغيّر حالة النظام.
- **Command:** بتطلب تغيير في حالة النظام، زي إضافة أو تعديل أو حذف موظف.

ليه نحتاج CQRS؟

في التطبيق التقليدي ممكن تلاقي Controller فيه القراءة والإضافة والتعديل والحذف، أو Service واحدة مسؤولة عن كل العمليات.

ده مش غلط بالضرورة، لكن مع كبر المشروع ممكن المسؤوليات تتداخل.

CQRS بيساعدك على:

1. فصل مسؤوليات القراءة عن الكتابة.
2. تنظيم الكود وتسهيل اختباره.
3. إضافة Validation و Logging و Authorization لكل عملية بشكل منظم.
4. تطوير عمليات القراءة والكتابة بشكل مستقل عند الحاجة.

طيب يعني إيه MediatR؟

MediatR هي مكتبة في .NET. بتساعدك تطبق نمط الـ Mediator.

خلينا نفهم المشكلة الأول.

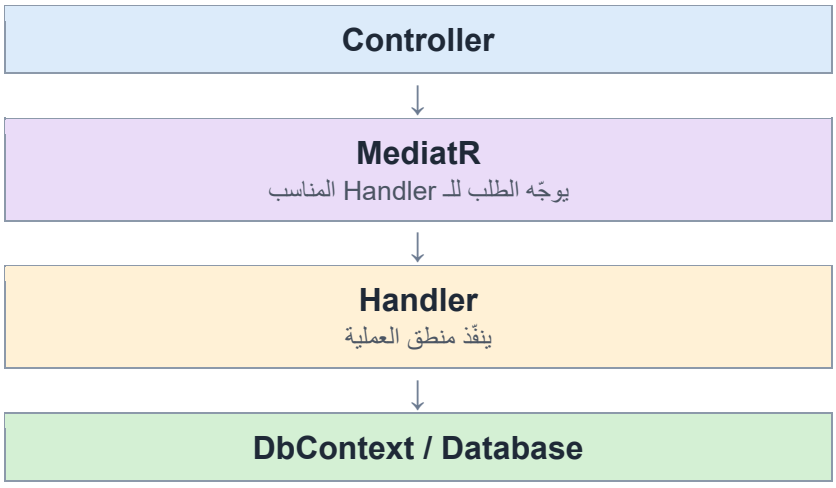
من غير MediatR، ممكن الـ Controller ينادي Service مباشرة:

```
[HttpPost]
public async Task<IActionResult> Create(CreateEmployeeDto dto)
{
    var employee = await _employeeService.CreateAsync(dto);

    return Ok(employee);
}
```

هنا الـ Controller مرتبط مباشرة بالـ Service.

مع MediatR، الـ Controller بيبعت Request، والـ Mediator بيوصّله للـ Handler المسؤول عنه.



الفرق المهم:

- **CQRS:** أسلوب لتنظيم وفصل عمليات القراءة والكتابة.
- **MediatR:** مكتبة بتساعدك توجّه الطلبات إلى الـ Handlers.

نفهم مكونات CQRS & MediatR

قبل الكود، لازم تفهم الأربع مصطلحات دول:

المصطلح	وظيفته
Command	يمثل طلب تغيير البيانات
Query	يمثل طلب قراءة البيانات
Handler	ينفّذ الطلب
IMediator / ISender	يرسل الطلب إلى الـ Handler المناسب.

شكل المشروع

ممكن تنظّم المشروع بالشكل ده:

```
EmployeeApi
│
├── Domain
│   └── Entities
│       └── Employee.cs
│
├── Application
│   └── Employees
│       ├── Commands
│       │   ├── CreateEmployeeCommand.cs
│       │   └── CreateEmployeeHandler.cs
│       │
│       └── Queries
│           ├── GetEmployeesQuery.cs
│           └── GetEmployeesHandler.cs
│
├── Infrastructure
│   └── Persistence
│       └── AppDbContext.cs
│
└── Controllers
    └── EmployeesController.cs
```

ده تنظيم توضيحي، وممكن في مشروع أكبر تفصل الـ Application والـ Infrastructure في مشاريع مستقلة.

إنشاء CreateEmployeeCommand

الملف: Application/Employees/Commands/CreateEmployeeCommand.cs

```
using MediatR;

public record CreateEmployeeCommand(
    string Name,
    decimal Salary
) : IRequest<int>;
```

خلينا نفهم كل جزء:

- `CreateEmployeeCommand`: اسم الطلب اللي بيمثل إضافة موظف.
- `Name` و Salary: البيانات المطلوبة.
- `IRequest<int>`: معناها إن الطلب متوقع يرجّع نتيجة من النوع int، وفي مثالنا هيرجّع رقم الموظف الجديد.

ليه استخدمنا Command مش EmployeeService؟

لأننا بنمثل عملية محددة، وهي إضافة موظف، بدل ما نجمع عمليات مختلفة في Service واحدة.

إنشاء الـ Handler

الملف: Application/Employees/Commands/CreateEmployeeHandler.cs

```
using MediatR;

public class CreateEmployeeHandler
    : IRequestHandler<CreateEmployeeCommand, int>
{
    private readonly AppDbContext _context;

    public CreateEmployeeHandler(AppDbContext context)
    {
        _context = context;
    }

    public async Task<int> Handle(
        CreateEmployeeCommand request,
        CancellationToken cancellationToken)
    {
        var employee = new Employee
        {
            Name = request.Name,
            Salary = request.Salary
        };

        _context.Employees.Add(employee);

        await _context.SaveChangesAsync(cancellationToken);

        return employee.Id;
    }
}
```

إيه اللي بيحصل هنا؟

1. الـ Handler يستقبل `CreateEmployeeCommand`.
2. ينشئ Employee من البيانات الموجودة في الطلب.
3. يضيفه إلى الـ DbContext.
4. يحفظ التغييرات في قاعدة البيانات.
5. يرجّع Id الموظف الجديد.

خد بالك: `Add()` لوحدها مش بتحفظ البيانات في قاعدة البيانات؛ الحفظ بيتم عند `SaveChangesAsync()`.

إنشاء الـ Controller

```
using MediatR;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/employees")]
public class EmployeesController : ControllerBase
{
    private readonly ISender _sender;

    public EmployeesController(ISender sender)
    {
        _sender = sender;
    }

    [HttpPost]
    public async Task<IActionResult> Create(
        [FromBody] CreateEmployeeCommand command,
        CancellationToken cancellationToken)
    {
        var id = await _sender.Send(command, cancellationToken);

        return Ok(new { Id = id });
    }
}
```

نطبّق Query لعرض الموظفين

هنا هنطبق الجزء التاني من CQRS: القراءة.

أولًا: إنشاء GetEmployeesQuery

الملف: Application/Employees/Queries/GetEmployeesQuery.cs

```
using MediatR;

public record GetEmployeesQuery()
    : IRequest<List<EmployeeDto>>;
```

هنستخدم DTO علشان نحدد البيانات اللي هنرجعها للعميل.

```
public record EmployeeDto(
    int Id,
    string Name,
    decimal Salary
);
```

ثانيًا: إنشاء Query Handler

الملف: Application/Employees/Queries/GetEmployeesHandler.cs

```
using MediatR;
using Microsoft.EntityFrameworkCore;

public class GetEmployeesHandler
    : IRequestHandler<GetEmployeesQuery, List<EmployeeDto>>
{
    private readonly AppDbContext _context;

    public GetEmployeesHandler(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<EmployeeDto>> Handle(
        GetEmployeesQuery request,
        CancellationToken cancellationToken)
    {
        return await _context.Employees
            .AsNoTracking()
            .Select(e => new EmployeeDto(
                e.Id,
                e.Name,
                e.Salary))
            .ToListAsync(cancellationToken);
    }
}
```

ليه استخدمنا `AsNoTracking()`؟

لأننا بنقرأ بيانات فقط ومش محتاجين EF Core يتابع التغييرات على الـ Entities. ده غالبًا يقلل تكلفة الـ Tracking في عمليات القراءة.

وليه استخدمنا `Select()`؟

علشان نرجع الحقول المطلوبة فقط بدل ما نرجع الـ Entity بالكامل.

ثالثًا: إضافة Endpoint للقراءة

داخل `EmployeesController`:

```
[HttpGet]
public async Task<IActionResult> GetAll(
    CancellationToken cancellationToken)
{
    var employees = await _sender.Send(
        new GetEmployeesQuery(),
        cancellationToken);

    return Ok(employees);
}
```

كده عندنا مسارين منفصلين:

- `POST /api/employees`: بيبعت Command لـ Handler علشان يضيف موظف.

- `GET /api/employees`: بيبعت Query لـ Handler علشان يقرأ الموظفين.

وده التطبيق الأساسي لفكرة CQRS.

إمتى أستخدم CQRS وإمتى أتجنبه؟

مش كل مشروع محتاج CQRS مع MediatR.

الحالة	القرار المناسب
CRUD بسيط جدًا	Controller و Service ممكن يكونوا كفاية
مشروع متوسط وفيه عمليات كثيرة	فصل كل عملية إلى Command أو Query قد يساعد
Business Logic معقد	CQRS يساعد على تنظيم العمليات واختبارها
عمليات قراءة ضخمة وتقارير	ممكن تفصل نموذج القراءة عن الكتابة عند الحاجة
Microservices	CQRS خيار معماري، لكنه مش شرط لكل Service

Task

بناء Products API باستخدام Clean Architecture And CQRS.

المتطلبات:

1. إنشاء Product يحتوي على Id, Name, Price.
2. تنفيذ `GET /api/products` و`GET /api/products/{id}` و`POST /api/products`.
3. فصل الـ Domain والـ Application والـ Infrastructure والـ API.
4. استخدام EF Core و SQL Server.
5. استخدام DTOs و Validation.